

3. Hardware and Virtual Machines

3.1. RISC & CISC Processors

- **RISC:** Reduced Instruction Set Computers.
- **CISC:** Complex Instruction Set Computers.

RISC	CISC
Fewer instructions	More instructions
Simpler instructions	Complicated instructions
A small number of instruction formats	Many instruction formats
Single-cycle instructions whenever possible	Multi-cycle instructions
Fixed-length instructions	Variable-length instructions
Only load and store instructions to address memory	Many types of instructions to address memory
Fewer addressing modes	More addressing modes
Multiple register sets	Fewer registers
Hard-wired control unit	Microprogrammed control unit
Pipelining easier	Pipelining much difficult

- **Pipelining:** Instruction level parallelism
 - Used extensively in RISC processor-based systems to reduce the time taken to run processes
 - Multiple registers are employed
- **Interrupt handling in CISC and RISC Processors:**
 - As soon as the interrupt is detected, the current processes are paused and moved into registers
 - The ISR (Interrupt Service Routine) is loaded onto the pipeline and is executed.
- When the interrupt has been serviced, the paused processes are resumed by bringing them back from the registers to the pipeline
- RISC processors allow for providing efficient pipelining. Pipelining is instruction-level parallelism. Its underlying principle is that the fetch-decode execute cycle can be separated into several stages.

One of the possibilities include:

1. Instruction fetch (IF)
2. Instruction decode (ID)
3. Operand fetch (OF)
4. Instruction execution (IE)
5. Result write back (WB)

Pipelining for Five-Stage Instruction Handling

		Clock cycles						
		1	2	3	4	5	6	7
Processor units	IF	1.1	2.1	3.1	4.1	5.1	6.1	7.1
	ID		1.2	2.2	3.2	4.2	5.2	6.2
	OF			1.3	2.3	3.3	4.3	5.3
	IE				1.4	2.4	3.4	4.4
	WB					1.5	2.5	3.5

- For pipelining to be implemented, the construction of the processor must have five independent units, each handling one of the five identified stages.
- Therefore, it's important for RISC processors to have many register sets. Each processor unit must have access to its own set of registers. The representations 1.1, 1.2, and so on are used to define the instruction and the stage of the instruction. Initially, only the first stage of the first instruction has entered the pipeline. At clock cycle 6, the first instruction has left the pipeline, the last stage of instruction 2 is being handled, and instruction 6 has just been entered. Once underway, the pipeline is handling 5 stages of 5 individual instructions. At each clock cycle, the complete processing of one instruction is finished. Without the pipelining, the processing time would've been 5 times longer.
- One disadvantage is interrupt handling. There will be 5 instructions in the pipeline when an interrupt occurs, so it needs to remove those previous instructions and then handle the interrupt.

3.2. Parallel Processing

- SISD**
 - Single Instruction Single Data stream
 - Found in the early computers
 - Contains a single processor; thus, there is no pipelining
- SIMD**
 - Single Instruction Multiple Data.
 - Found in array processors
 - Contains multiple processors, which have their own memory.
- MISD**
 - Multiple Instruction Single Data
 - Used to sort large quantities of data.
 - Contains multiple processors which process the same data
- MIMD**
 - Multiple Instruction Multiple Data.
 - Found in modern personal computers.
 - Each processor executes a different individual instruction.
- Massively parallel computers**
 - Computers that contain vast amounts of processing power.
 - Has a bus structure to support multiple processors and a network infrastructure to support multiple 'Host' computers.
 - Commonly used to solve highly complex mathematical problems.

3.3. Virtual Machines

- Virtual machine:**
 - Process interacts with the software interface provided by the OS. This provides an exact copy of the hardware.
 - OS kernel handles interaction with actual host hardware

Pros	Cons
Allows more than one OS to run on a system	Performance drop from native OS
Allows multiple copies of the same OS	The time and effort needed for implementation is high

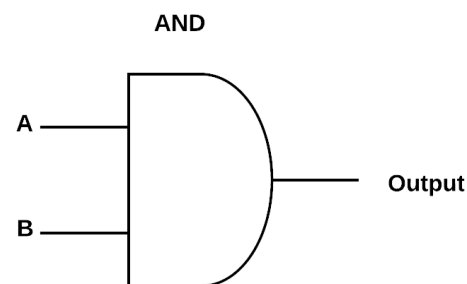
Examples and Usage:

- Used by companies wishing to use the legacy software on newer hardware and server consolidation companies
- Virtualising machines allows developers to test applications on many systems without making expensive hardware purchases.

3.4. Logic Gates & Circuit Design

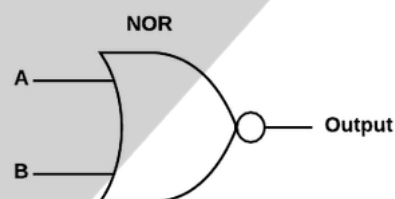
Logic Gates: A component of a logical circuit that can perform a Boolean operation

- AND Gate:** $A \cdot B = X$



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

- OR Gate:** $A + B = X$



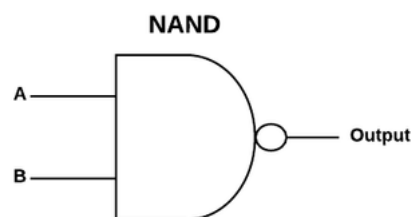
A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

- NOT Gate:** $\bar{A} = X$



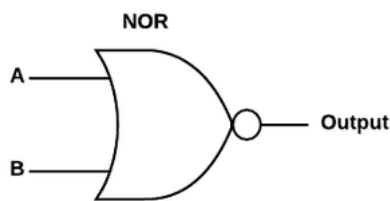
A	Output
0	1
1	0

- NAND Gate:** $\overline{A \cdot B} = X$



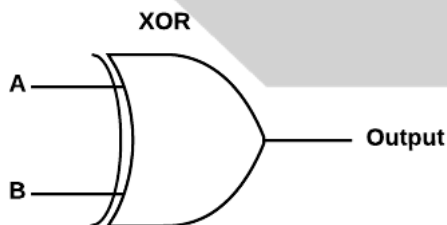
A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

- NOR Gate: $\overline{A + B} = X$



A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

- XOR Gate: $A \cdot \overline{B} + \overline{A} \cdot B = X$



A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Half-Adder

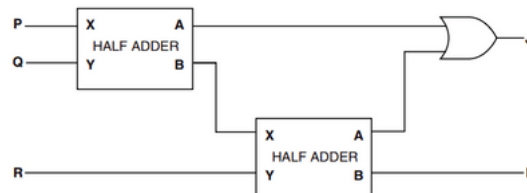
- A half adder adds two binary digits and outputs the sum (s) and their carry (c).

- A XOR gate acts like a half adder.

Input		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full-Adder

A full adder adds three binary digits and outputs the sum and their carry.



- R is "Carry In"
- J is "Carry Out"
- K is "Sum"

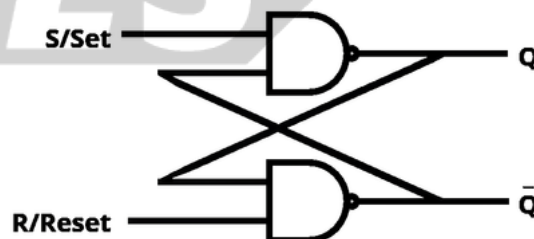
Input			Output	
P	Q	R	J	K
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

3.5: Flip-Flops

- Flip flops can store a single bit of data as 0 or 1
- Computers use bits to store data.
- Flip-flops can be used to store bits of data.
- Memory can be created from flip-flops.

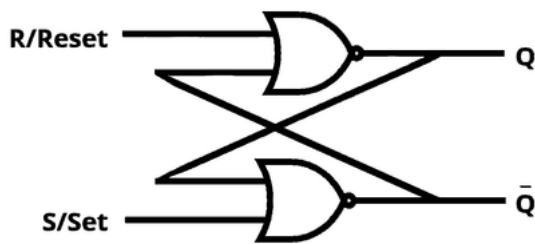
SR Flip Flops

- Can be made with NAND gates or NOR gates
- Error state exists



S	R	Q	Q Bar
0	0	1	1
0	1	1	0
1	0	0	1
1	1	No Change	No Change

- 00 causes an error as Q Bar is no longer the complement of Q
- 01 sets Q to 1
- 10 resets Q to 0
- 11 leads to no change; the initial value remains

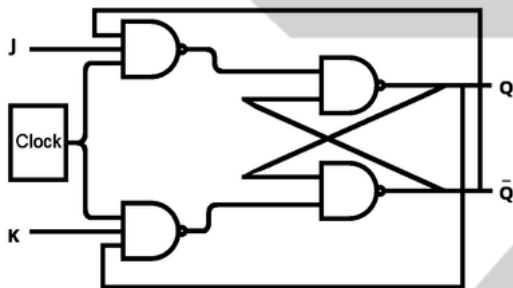


S	R	Q	Q Bar
0	0	No Change	No Change
0	1	0	1
1	0	1	0
1	1	0	0

- 11 causes an error as Q Bar is no longer the complement of Q
- 10 sets Q to 1
- 01 resets Q to 0
- 00 leads to no change; the initial value remains

JK Flip Flops

- JK flip flops are only active when the clock is high.
- JK flip flops use a clock pulse for synchronization to ensure proper functioning.
- Advantages of JK flip flops include the validity of all input combinations, avoidance of unstable states, and increased stability compared to SR flip flops.



Clock	J	K	Q	Q Bar
0	0	0	-	-
0	0	1	-	-
0	1	0	-	-
0	1	0	1	1
1	0	0	No Change	No Change
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

3.6. Boolean Algebra

- Double Complement: $\overline{\overline{A}} = A$
- Identity Law
 - $1 \cdot A = A$
 - $0 + A = A$
- Null Law
 - $0 \cdot A = 0$
 - $1 + A = 1$
- Idempotent Law
 - $A \cdot A = A$
 - $A + A = A$
- Inverse Law
 - $A \cdot \overline{A} = 0$
 - $A + \overline{A} = 1$
- Commutative Law
 - $A \cdot B = B \cdot A$
 - $A + B = B + A$
- Associative
 - $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
 - $(A + B) + C = A + (B + C)$

- Distributive Law
 - $A + B \cdot C = (A + B) \cdot (A + C)$
 - $A \cdot (B + C) = A \cdot B + A \cdot C$
- Absorption Law
 - $A \cdot (A + B) = A$
 - $A + A \cdot B = A$
- De Morgan's Law
 - $\overline{(A \cdot B)} = \overline{A} + \overline{B}$
 - $\overline{(A + B)} = \overline{A} \cdot \overline{B}$

(example)

(c) Simplify the following expression using Boolean algebra, including De Morgan's laws. Show your working.

Working: $\overline{(A \cdot B)} \cdot (A \cdot C) \cdot (B \cdot D)$

$\overline{A \cdot B} + \overline{A \cdot C} + \overline{B \cdot D}$ De Morgan's Law

$\overline{A} \cdot \overline{B} + \overline{A} \cdot \overline{C} + \overline{B} \cdot \overline{D}$ Cancelling out 2 - cause NOT and NOT cancel out

$\overline{A} \cdot (\overline{B} + \overline{C}) + \overline{B} \cdot \overline{D}$ Distributive law (backwards)

Answer: $\overline{A} \cdot (\overline{B} + \overline{C}) + \overline{B} \cdot \overline{D}$

3.7. Karnaugh Maps

- **Karnaugh maps:** a method of obtaining a simplified Boolean algebra expression from a truth table. They minimize the number of logic circuits, thus making it more efficient.
- **Sum of products:** all the combinations that lead to 1 in the output.

Methodology

- Try to look for trends in the output, thus predicting the presence of a term in the final expression
- Draw out a Karnaugh Map by filling in the truth table values into the table
- Select groups of '1' bits in even quantities (2, 4, 6, etc.); if not possible, then consider a single input as a group
- Note: Karnaugh Maps wrap around columns
- Within each group, only the values that remain constant are retained

Example

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

1. Sum of Products

$$A \cdot \overline{B} \cdot \overline{C} + A \cdot B \cdot \overline{C} + A \cdot B \cdot C$$

2. Fill in the map

		AB			
		00	01	11	10
C	0	0	0	1	1
	1	1	0	0	1

3. Circle the largest group containing an even number of 1s.

		AB			
		00	01	11	10
C	0	0	0	1	1
	1	1	0	0	1

4. Take the common values from both

- In the first group, $A \cdot B$ is common
- In the second group, $A \cdot \overline{C}$ is common

CAIE A2 LEVEL COMPUTER SCIENCE

5. Write the simplified expression.

$$A \cdot \overline{C} + A \cdot B$$